

Introduction to Unsupervised Learning

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Distinguish supervised, unsupervised, semi-supervised, and self-supervised learning based on what data is available.
- Explain why some domains, such as biomedical and legal data, struggle to produce labeled data.
- Apply distance-based intuition (within-cluster and across-cluster distance) to judge whether data points form genuine clusters.
- Describe how k-means clustering formalizes this intuition, including how to choose k and how anomalies can surface as a byproduct.

2. Detailed Explanation

Categories of machine learning: supervised vs. unsupervised

Machine learning algorithms fall into two primary classes: supervised and unsupervised. A third label, semi-supervised, has become common in recent years. It is not a fundamentally different category — rather, it is a practical strategy that blends supervised and unsupervised techniques instead of forming a new algorithm family. Which category applies depends entirely on the data available: whether both input (X) and output labels (Y) exist, or only X exists.

In supervised learning, every input record has an associated output label, so the data comes in pairs of the form (X, Y) . The label carries extra information that tells the algorithm how to adjust its internal parameters to improve performance. A good

example is sentiment classification: product reviews from e-commerce sites such as Amazon or Flipkart naturally come labeled, because reviewers attach a star rating alongside their written review. The review text becomes the input (X), and the star rating becomes the output label (Y). A common simplification maps ratings of three stars and above to a positive sentiment label and ratings below three stars to a negative label, turning the problem into binary classification. The same data could instead be treated as a five-class multi-class classification by keeping each star value (one through five) as its own label.

Classification problems more generally predict a discrete category from an input. Spam detection classifies an email as spam, legitimate, or primary. Weather-based prediction classifies whether a person will go out to play based on weather parameters. In each case, a labeled dataset teaches the algorithm the relationship between input features and the discrete output category. Supervised algorithms include linear regression, logistic regression, decision trees, random forests, and neural networks (including deep neural networks for classification).

Why some domains cannot easily produce labels, and what annotation means

Some domains cannot easily produce the Y label even though the input X is abundant. Legal and biomedical domains are two examples. Annotation (also called labeling) is the process of assigning a class or output value to a data point that otherwise only has input features available. Domains where labels are easy to obtain, such as star-rating-based sentiment classification, favor supervised learning because annotation is cheap. Domains where labels are expensive or impossible to obtain at scale push practitioners toward unsupervised learning.

Consider tumor classification. A hospital or research agency might collect measurements for many patients, such as age and an activity score describing how

physically active a patient is, independent of age. These measurements are collected along with the type of tumor observed (benign or malignant). Malignant tumors are more often observed among elderly women, and activity level tends to decrease with age. Because of this relationship, younger, more physically active women tend to cluster with benign tumor cases, and older, less active women tend to cluster with malignant tumor cases. In many real situations, only age and activity score (X) are available; the tumor type (Y) is missing entirely. Doctors are the only experts qualified to make the diagnosis, but they are too occupied with patient care to manually annotate large volumes of data. Radiologists face a similar limitation. They can describe an observation, such as "a bigger lump is observed in this part of the breast," but the final diagnostic verdict rests with the doctor, whose time for annotation work is scarce.

The legal domain has the same problem. Identifying the statute or the verdict inside a legal document requires an expert, typically a lawyer, and legal language is often too complex for a non-expert to interpret. Hiring an expert purely to annotate documents is expensive, since it pulls that expert away from their core work for what is effectively a clerical task.

Moving to unsupervised learning: distance as the primary signal

When only X is available and Y is missing, the objective changes. Instead of learning a function that maps X to Y , the goal becomes finding patterns in the data itself: groups of records that are similar to each other and distinct from other groups. Proximity between data points, measured through distance, is what makes pattern-finding possible without labels.

Two distance types are commonly used:

- Euclidean distance: the direct, straight-line distance between two points. Picture it as the "walking distance" you'd take moving diagonally toward someone across a room, rather than following a rectangular path along the walls.
- Cosine distance: measures similarity based on the angle between vectors, and is used more often when comparing vector representations.

Clusters, within-cluster distance, and across-cluster distance

A cluster is a group of data points that sit close to one another when distance is computed between them. If a dataset contains two underlying classes, plotting the data tends to reveal two visually distinct groupings, without needing to know the class labels in advance.

Two quantities describe how distinct these groupings are:

- Within-cluster distance: the average pairwise distance between data points that belong to the same cluster.
- Across-cluster distance: the average pairwise distance between data points that belong to different clusters.

If the ratio of average across-cluster distance to average within-cluster distance is high, the data points genuinely form separate clusters. A large ratio means the numerator (across-cluster distance) is far bigger than the denominator (within-cluster distance), which signals well-separated groups. Once a cluster is identified, checking the true label of just one data point inside it is often enough, since the rest of the cluster is assumed to share that same label.

Watch out for: uneven cluster sizes do not break this calculation. If one cluster contains more data points than another, the within-cluster and across-cluster distances are still computed correctly, because each is an average divided by the actual number of pairs

involved. A larger cluster simply divides by a larger number of pairs; it does not distort the comparison.

K-means clustering, choosing k, and what it means to be a multi-class classifier

K-means clustering is the algorithm that formalizes the distance-based intuition above. The name reflects two ideas: "means" refers to computing average (mean) distances, and "k" refers to the number of clusters the algorithm is asked to find. K-means forms k clusters by finding k centroids. A centroid is the average position of all the data points assigned to a cluster, and the algorithm pulls nearby data points toward the centroid they are closest to.

K is a hyperparameter that must be chosen before running the algorithm. A value of k equal to one does not make sense, since that would not represent a classification problem at all. A common approach is to try $k = 2, 3, 4$, and so on, and compare performance against a labeled validation dataset. Testing multiple discrete values of a hyperparameter one at a time, rather than sampling values randomly, is called grid search. Tuning k this way is a simple case of grid search because only one hyperparameter is being tuned.

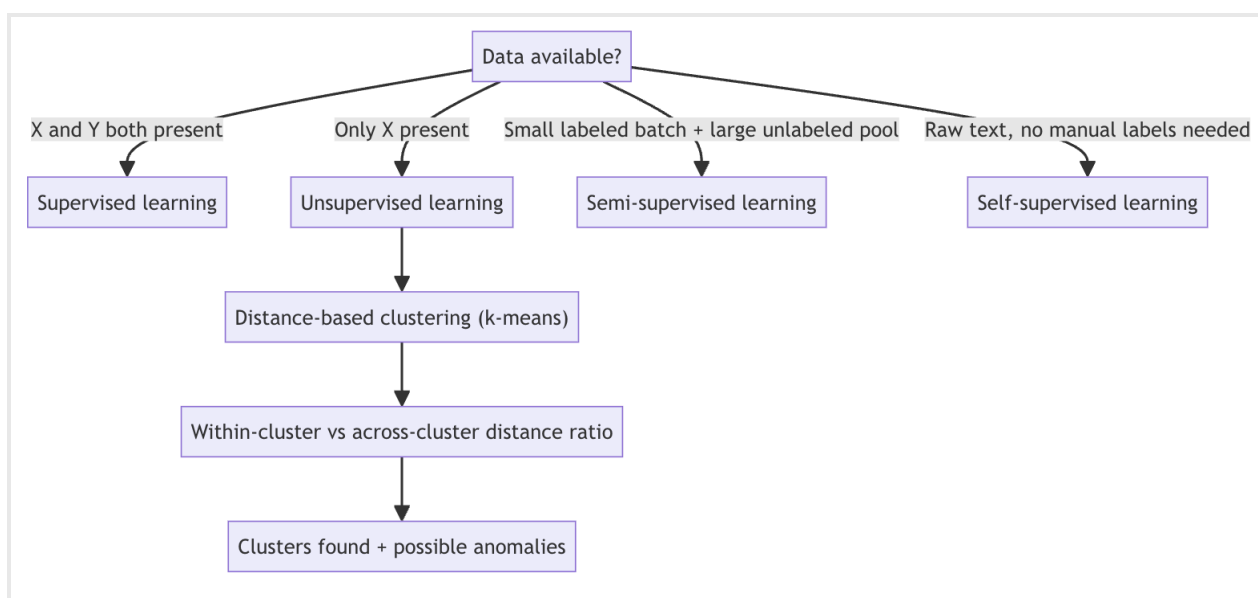
Consider t-shirt sizing: an e-commerce platform might have a large batch of t-shirts, each described only by length and width, with no size label attached (small, medium, large, or extra-large). If the manufacturing factory is known to produce exactly four sizes, that gives a strong hint that k should be set to four. Running k-means with $k = 4$ on the length and width features produces four clusters: larger length-and-width combinations form the extra-large group, then large, then medium, then small. Because width does not shrink or expand in exact proportion to length across different body types, the clusters do not fall neatly along a single straight line. If k were set too low, for example $k = 2$, the algorithm would merge distinct groups (such as extra-large with

small) into the same cluster, producing a worse fit. The correct k is confirmed by checking classification accuracy on a labeled validation set: as k increases from two toward the true number of groups, misclassification drops noticeably, and increasing k further does not help.

K-means clustering is a multi-class classifier by default: it can identify any number of groups (k can be set higher than two), unlike a strictly binary classifier.

Clustering can also surface anomalies as a byproduct, even though anomaly detection is not an explicit part of the algorithm. If a single data point, or a very small handful of points, ends up far from every other cluster, it forms its own tiny cluster and can be flagged as an anomaly. In the t-shirt example, a t-shirt manufactured with the wrong dimensions would show up as an isolated point far from the other clusters, revealing a defect. In practice, a workflow can run k-means clustering first, discard the points that form outlier clusters, and then train a supervised algorithm (such as logistic regression or a support vector machine) on the cleaned data.

The diagram below summarizes how the choice of data available routes you toward each learning category:



Why supervised learning is preferred when labels are available

Unsupervised learning is never the preferred choice when both X and Y are available. It is a "something is better than nothing" fallback used only when Y cannot be obtained.

Both theoretical and experimental evidence show that unsupervised algorithm performance is never comparable to supervised algorithm performance on the same task. This is because clustering approaches have no access to the true label and therefore no direct signal to adjust their internal parameters against.

An analogy illustrates why labels matter. Suppose a person is shown many transcripts of debates (such as a debate about a farmers' protest), each paired with its final conclusion. That person becomes better at predicting the outcome of a new, unseen debate. If the same person is shown only the debate content without the conclusion, they have far less to learn from. Machine learning algorithms behave the same way. Given X and Y together, an algorithm learns how to adjust its parameters toward the correct answer; given X alone, it has no such target to learn against. Once enough labeled (X, Y) data becomes available, switching to a supervised algorithm is always the right move. A team might start with an unsupervised approach when labels are scarce, then deliver an upgraded supervised model once sufficient annotated data has accumulated.

Worked example: a synthetic music-listener dataset

A synthetic dataset built in a Python notebook demonstrates the distance-based clustering intuition. It describes listeners using two features: average song length and skip rate (how often a listener skips a song before it finishes). The idea draws on a real listening pattern. Listeners who choose longer content, such as long-form recipe or speech videos, tend to have a higher skip rate, jumping ahead once they judge the content too long. Listeners who choose very short content, such as short-form reels or status videos (capped at roughly 90 seconds on platforms like Instagram or WhatsApp),

tend to watch to completion and have a lower skip rate. The same pattern was framed in musical genre terms. Sufi songs and ghazals tend to run long, roughly seven to twelve minutes with three or four stanzas. Many Bollywood rap songs are much shorter, as short as about two and a half minutes with as few as two stanzas, and move quickly. The dataset does not include a genre label; the goal is to see whether genre-like groupings emerge purely from the two numeric features. These genre-length claims are illustrative, not universal – exceptions exist in both directions, including shorter ghazals.

```
import numpy as np
import pandas as pd
from scipy.spatial.distance import pdist, squareform

# Group A: shorter, low-skip-rate listeners
group_a = np.random.normal(
    loc=[3, 0.15], scale=[0.3, 0.05], size=(4, 2)
)

# Group B: longer, high-skip-rate listeners
group_b = np.random.normal(
    loc=[7.5, 0.7], scale=[0.3, 0.05], size=(4, 2)
)

listeners = np.round(np.vstack([group_a, group_b]), 2)
listeners_df = pd.DataFrame(
    listeners,
    columns=["average song length", "skip rate"],
    index=[chr(65 + i) for i in range(8)],
)
```

Group A was generated with a mean song length of 3 (minutes) and a mean skip rate of 0.15, using standard deviations of 0.3 for song length and 0.05 for skip rate. Group B

was generated with a much higher mean song length of 7.5 and mean skip rate of 0.7, using the same standard deviations as group A. Each group produced a 4-by-2 matrix (four listeners, two features). `numpy`'s `vstack` function stacked the two matrices vertically into a single 8-by-2 matrix of eight listeners, with values rounded to two decimal places (for example, 2.98 and 8.14). The eight rows went into a pandas DataFrame indexed A through H, generated from ASCII character codes (65 converts to "A," 66 to "B," and so on). Plotting the listeners showed two visually distinct groupings: listeners A through D clustered in the short-song, low-skip-rate region, and listeners E through H clustered in the long-song, high-skip-rate region.

```
distances = squareform(pdist(listeners))
distance_df = pd.DataFrame(
    np.round(distances, 2),
    index=listeners_df.index,
    columns=listeners_df.index,
)
```

`pdist`, from `scipy.spatial.distance`, computes the pairwise Euclidean distance between every pair of the eight listeners. `squareform` converts that pairwise output into a full 8-by-8 distance matrix, where the entry in row A, column B is the distance between listener A and listener B. The diagonal is zero (distance from each point to itself), and the matrix is symmetric. The 8-by-8 matrix naturally splits into four blocks: within-group A, within-group B, and two identical across-group blocks. Distances inside the within-group blocks were small, roughly 0.24 to 0.33. Distances inside the across-group block were much larger, roughly 4.8 to 5.2.

```
within_a = distances[:4, :4][np.triu_indices(4, k=1)]
within_b = distances[4:, 4:][np.triu_indices(4, k=1)]
across = distances[:4, 4:].flatten()

avg_within = np.concatenate([within_a, within_b]).mean()
```

```
avg_across = across.mean()
ratio = avg_across / avg_within
```

`np.triu_indices` extracts only the upper triangle of a sub-matrix, avoiding double-counting each pair. Slicing `distances[:4, :4]` isolates group A, `distances[4:, 4:]` isolates group B, and `distances[:4, 4:]` isolates the across-group block, flattened since every value there is a genuine across-group distance. Result: the average within-cluster distance was 0.355, and the average across-cluster distance was 4.69. Dividing 4.69 by 0.355 gives a ratio of about 13.2 — the across-cluster distance is roughly 13 times larger than the within-cluster distance. This is a strong signal that the eight listeners genuinely form two separate clusters rather than one uniform group. K-means clustering performs this same distance computation internally; the only difference is that the analyst supplies the value of *k* up front, and the algorithm forms that many clusters automatically.

Worked example: the Iris flower dataset

The Iris dataset is a well-known dataset containing 150 flower records, with four numeric features: sepal length, sepal width, petal length, and petal width. It covers three species of Iris flowers, making it a multi-class classification dataset, and it ships as a built-in dataset inside scikit-learn's `datasets` module.

```
from sklearn.datasets import load_iris

iris = load_iris()
X_iris = iris.data
y_iris = iris.target # not used in this walkthrough

print(X_iris.shape) # (150, 4)
```

`load_iris` returns an object with `.data` (the X features) and `.target` (the Y species labels). `x_iris.shape` returns `(150, 4)`: 150 flower instances (rows) and 4 features (columns). The target labels were intentionally not used, since the goal was to demonstrate clustering behavior on unlabeled features rather than to train a classifier. Since only two dimensions can be shown on a flat plot, sepal length (column 0) and petal length (column 2) were plotted against each other, using a dot size of 30 and semi-transparent circles. Plotting just these two features already revealed two visually distinct groupings, even though the full dataset contains three species; using more of the four available features makes the separation between the three species clearer. The Iris dataset is popular for teaching classification because a supervised algorithm trained on both X and Y typically reaches about 95% accuracy. A stronger algorithm, such as a neural network, can push that up to about 99% accuracy.

Semi-supervised learning

Semi-supervised learning is not a separate algorithm. It is a practical strategy for combining a small, carefully hand-annotated dataset with a much larger pool of unlabeled data, applied in stages using ordinary supervised algorithms. Suppose 150 Iris records are manually and accurately annotated, alongside a much larger pool of roughly 10,000 additional unlabeled instances. The process works as follows:

1. Manually annotate a small, manageable batch (for example, the 150 Iris records) and train an initial supervised model, version 1, such as logistic regression.
2. Use version 1 to predict labels for a further small batch of previously unlabeled data (for example, 100 instances).
3. Add that newly labeled batch back into the training set (now roughly 250 instances) and retrain to produce version 2.
4. Repeat with additional batches (for example, another 200 instances), producing version 3 and beyond, each time retraining on the growing labeled set.

5. Hold out a separate labeled test set (for example, 1,000 instances) to measure whether each new version improves on the last.

Predictions generated by an earlier version will not be perfectly accurate. An early round might see about 80% of a newly labeled batch of 100 instances turn out correct and about 20% incorrect. A later round might improve to roughly 90% correct and 10% incorrect. Even with this imperfection, the final semi-supervised model typically performs better than the original model trained only on the small, purely hand-annotated set. The motivation is practical: manually annotating 150 records is realistic for an expert to complete in a couple of hours, but manually annotating 10,000 or more records is repetitive manual labor. Most experts are unwilling to take on such a task since it pulls them away from the judgment-driven work they are trained for.

Self-supervised learning

Self-supervised learning is a separate concept from semi-supervised learning and applies mainly to language models, including large language models such as GPT-family models and other Indic-language models. It does not require explicit manual annotation at all. Given a window of consecutive words in raw text — for example, the nine words preceding a target word — a language model is trained to predict the next word. That predicted word, word number ten, completes the window. Because the "answer" (the actual next word) is already present in the raw text, no human needs to manually annotate it. As the window slides across a large body of text, each step produces a new (X, Y) training pair automatically: X is the preceding words, and Y is the word that actually follows them.

This is why progress in language models has been rapid: raw text data is abundant and requires no manual labeling. Large technology companies have access to vast amounts of text through emails, documents, and social media posts that people share publicly or

store on cloud platforms. Subject to their policies, this data can be used to train increasingly capable language models.

3. Key Takeaways

- The learning category you use depends on what data you have. (X, Y) pairs point to supervised learning; X alone points to unsupervised learning; a small labeled batch plus a large unlabeled pool points to semi-supervised learning; and raw text that supplies its own answer points to self-supervised learning.
- Unsupervised learning is a fallback, not a preference — it is used only when labels (Y) are too expensive or impossible to obtain, as in biomedical and legal domains, and its performance never matches supervised learning on the same task.
- Distance (Euclidean or cosine) is the core signal for finding structure without labels: a high ratio of across-cluster distance to within-cluster distance means the data genuinely forms separate groups.
- K-means clustering formalizes this intuition by finding k centroids and pulling nearby points toward them; k is a hyperparameter tuned via grid search, and isolated points left over from clustering can reveal anomalies.
- Semi-supervised learning bootstraps a small labeled set into a larger one through repeated rounds of prediction and retraining, while self-supervised learning (used for language models) generates its own labels directly from raw text.

Mental model: think of unsupervised learning as sorting a box of mixed items by feel alone (size, weight, texture) when nobody has told you what the items are. You group what's close together and call each tight group a cluster. Then you bring in an expert only to name a group once, rather than to name every single item.